Experiment No. 2

Experiment No. 01

Aim: Implementation of Multilayer Perceptron algorithm to simulate XOR gate.

Theory:

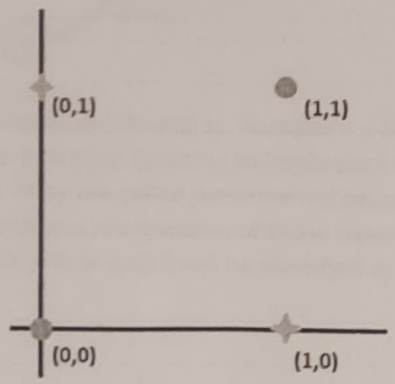
XOR, or Exclusive OR, is a binary logical operator that takes in Boolean inputs and gives out True if and only if the two inputs are different. This logical operator is especially useful when we want to check two conditions that can't be simultaneously true. The following is the Truth table for XOR function.

Input 1	Input 2	Output
0	0	0
0	1	1
1	0	1
1	1	0

The XOR problem:

The XOR problem is that we need to build a Neural Network (a perceptron in our case) to produce the truth table related to the XOR logical operator. This is a binary classification problem. Hence, supervised learning is a better way to solve it. In this case, we will be using perceptrons. Uni layered perceptrons can only work with linearly separable data. Linear separability of points is the ability to classify the data points in the hyperplane by avoiding the overlapping of the classes in the planes. Each of the classes should fall above or below the separating line and then they are termed as linearly separable data points. With respect to logical gates operations like AND or OR the outputs generated by this logic are linearly separable in the hyperplane.

But in the following diagram drawn in accordance with the truth table of the XOR logical operator, we can see that the data is **NOT linearly separable.** Since straight line drawn in any direction can not easily separate orange and green points without overlap and hence no linear boundary can be defined.



perceptrons cannot be used for XOR logic using the outputs generated by the XOR logic and the corresponding graph for XOR logic as shown below.

The Solution:

To solve this problem, we add an extra layer to our perceptron, i.e., we create a Multi Layered Perceptron (or MLP). We call this extra layer as the Hidden layer. To build a perceptron, we first need to understand that the XOR gate can be written as a combination of AND gates, NOT gates and OR gates in the following way:

Let a and b are input variables and a' and b' represents 'NOT a' and 'NOT b' respectively. Further AND can be represented by ' . ' operator and OR can be represented by ' + ' operator.

a XOR b = (a AND (NOT b)) OR (b AND (NOT a)) = a.b' + a'.b

= a.b' +a'.b + a.a' + b.b'

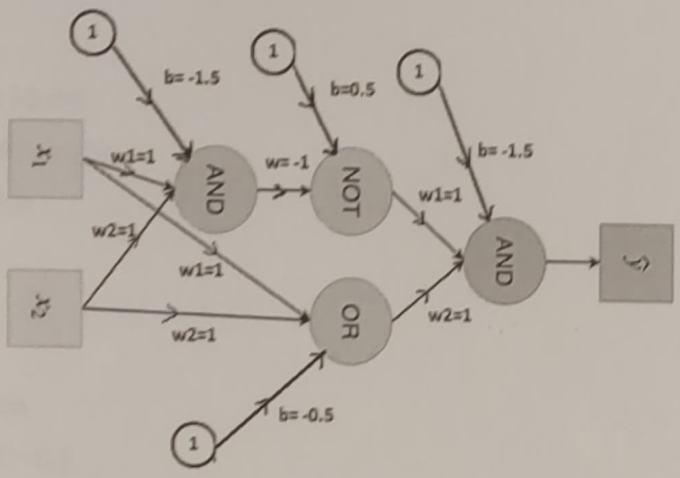
= a . (b' +a') + b.(a'+ b') = (a'+ b') . (a + b)

Now using De morgan's law in the first term,

= (a'' . b'')' . (a + b) = (a . b)' . (a + b)

a XOR b = (a . b)' . (a + b)

The following is a plan for the perceptron.



Here, we need to observe that our inputs are 0s and 1s. To make it a XOR gate, we use individual single perceptron with a binary step activation function to implement each one of the fundamental logical functions: NOT, AND and OR. They are called *fundamental* because any logical function, no matter how complex, can be obtained by a combination of those three. The threshold for classification would be 0.5, i.e., any x with σ(x)>0.5 will be classified as 1 and others will be classified as 0.

Conclusion:

```
# -*- coding: utf-8 -*-
"""
Created on Wed Jul 12 16:55:35 2023
@author: tiver
"""
# importing Python library
import numpy as np
# define Unit Step Function
def unitStep(v):
    if v >= 0:
        return 1
    else:
        return 0

# design Perceptron Model
def perceptronModel(x, w, b):
    v = np.dot(w, x) + b
    y = unitStep(v)
    return y

# NOT Logic Function
# wNOT = -1, bNOT = 0.5
def NOT_logicFunction(x):
    wNOT = -1
    bNOT = 0.5
    return perceptronModel(x, wNOT, bNOT)

# AND Logic Function
# here w1 = wAND1 = 1,
# w2 = wAND2 = 1, bAND = -1.5
def AND_logicFunction(x):
```

```
    w = np.array([2, 1])
    bAND = -3
    return perceptronModel(x, w, bAND)

# OR Logic Function
# w1 = 1, w2 = 1, bOR = -0.5
def OR_logicFunction(x):
    w = np.array([1, 1])
    bOR = -1
    return perceptronModel(x, w, bOR)

# XOR Logic Function with AND, OR and NOT
# function calls in sequence
def XOR_logicFunction(x):
    y1 = AND_logicFunction(x)
    y2 = OR_logicFunction(x)
    y3 = NOT_logicFunction(y1)
    final_x = np.array([y2, y3])
    finalOutput = AND_logicFunction(final_x)
    return finalOutput

# testing the Perceptron Model
test1 = np.array([0, 1])
test2 = np.array([1, 1])
test3 = np.array([0, 0])
test4 = np.array([1, 0])

print("XOR({}, {}) = {}".format(0, 1, XOR_logicFunction(test1)))
print("XOR({}, {}) = {}".format(1, 1, XOR_logicFunction(test2)))
print("XOR({}, {}) = {}".format(0, 0, XOR_logicFunction(test3)))
print("XOR({}, {}) = {}".format(1, 0, XOR_logicFunction(test4)))
```